

Pulseboard breaking-point campaign

A controlled extension of the V1/V2 realistic benchmark, using raised test-only connection caps and clean Redis state to distinguish per-room fan-out limits from total server WebSocket capacity.

Metric	V1	Corrected V2
Largest passing single room	210 users	225 users
First failing single-room probe	225 users	235 users
Largest stable distributed run	1,000 users / 50 rooms	600 users / 30 rooms
First failing distributed probe	1,200 users / 60 rooms	700 users / 35 rooms
Peak passing operation rate	395.0 ops/s	240.5 ops/s

What changed from the first report

The original suite stopped at 200 users, so it validated a ceiling without finding failure. This campaign probed beyond that ceiling. A run counted as passing only when latency stayed below 150 ms p95 and 300 ms p99, initial connections exceeded 99%, and clients remained stable without a replacement-upgrade storm.

The 200-user V2 repeat was reproducible: p95 56 ms and p99 86 ms, close to the original 49/81 ms. The sharp degradation above 225 users is therefore a real nonlinear fan-out knee, not evidence that the 200-user result was fabricated or anomalous.

Headline conclusion

V2 supports about 7% more users in one intensely shared room, but V1 sustains substantially more users when load is distributed across small rooms. V2's bounded slow-client queues begin evicting clients around 700 distributed users on this VM; V1 accepted more backlog and remained stable through 1,000 users during the 30-second window.

Fan-out creates a sharp latency cliff

Probe	Stable checks	Ops/s	p50 ms	p95 ms	p99 ms	Result
V1 210	100.0%	82.5	15	66	108	PASS
V1 225	100.0%	93.8	29	195	413	FAIL
V1 250	100.0%	93.2	532	2227	3233	FAIL
V1 300	100.0%	88.1	4178	10863	14418	FAIL
V1 400	100.0%	96.2	9822	18316	20692	FAIL
V2 200 repeat	100.0%	79.5	12	56	86	PASS
V2 225	100.0%	89.7	30	145	249	PASS
V2 235	100.0%	91.7	64	507	906	FAIL
V2 250	100.0%	93.9	558	2913	4370	FAIL
V2 300	100.0%	100.4	3869	11143	15447	FAIL
V2 400	100.0%	95.6	9876	17916	20016	FAIL

Why 200 succeeds while 250 collapses

The workload grows faster than the user count. More participants create more active strokes, and each stroke is broadcast to more recipients. Moving from 200 to 250 users increases the approximate sender-recipient work by $(250/200)^2 = 1.56$, or about 56%. Once socket queues accumulate, latency feeds back into slow-client drops and reconnection attempts.

V1 passed at 210 (66 ms p95, 108 ms p99) but failed at 225 (195/413 ms). V2 passed at 225 by a narrow margin (145/249 ms) and failed at 235 (507/906 ms). These are operational boundaries for this workload and VM, not immutable constants.

Reproducibility caveat

The probabilistic workload is deterministic per virtual-user seed, but exact scheduling, connection ramp timing, Go runtime pauses, Linux networking, and B-series CPU-credit state still vary. One short sample per boundary cannot establish a confidence interval. The healthy 200-user result was repeated; the exact 210-vs-225 and 225-vs-235 boundaries should be confirmed with 3-5 repetitions before making a production SLA.

Distributed rooms scale much further

Probe	Stable checks	Extra attempts	Ops/s	p95 ms	p99 ms	Result
V1 600	100.0%	0	238.7	10	33	PASS
V1 800	100.0%	0	318.2	30	66	PASS
V1 1000	100.0%	0	395.0	75	117	PASS
V1 1200	75.4%	411	435.3	131	211	FAIL
V1 1500	95.4%	76	549.2	257	412	FAIL
V2 400	100.0%	0	159.7	6	12	PASS
V2 600	100.0%	0	240.5	7	27	PASS
V2 700	79.2%	193	270.0	22	46	FAIL
V2 800 clean	62.4%	506	318.8	38	70	FAIL

Why the earlier V2 800 result was retested

The first V2 800-user run followed several destructive overload probes and showed instability. A fresh server and clean Redis repeat produced the same failure pattern, with 506 replacement upgrades. This confirms that V2's 800-user failure is reproducible. V2 also failed cleanly at 700 with 193 replacement upgrades, while 600 completed with no extras.

Measurement bug discovered

The existing custom `websocket_connection_success` metric counts successful open events. A client that opens, is dropped, and then retries can therefore still report 100% even while the service is unstable. The k6 websocket-upgrade checks exposed the replacement storm. Future harness versions should threshold both upgrade-attempt success and full expected-lifetime completion.

Interpretation

V1 is faster at distributed capacity in this short test, but its larger effective backlog may hide growing memory and latency during long holds. V2 fails earlier by design because it disconnects slow clients when bounded queues fill. The next backend improvement should coalesce cursor events and batch/fan-out writes so V2 preserves its safety properties without evicting healthy clients at 700+ connections.

All VMs were deallocated after artifact download. Raw k6 JSON/text logs, Prometheus snapshots, Docker samples, server logs, and run metadata are stored under `benchmark-results/20260912T1706Z-breakpoint`.